Name: Prathamesh Sunil Lad

1. SOC Fundamentals and Operations

A Security Operations Center (SOC) is a centralized unit responsible for monitoring, detecting, and responding to cybersecurity threats within an organization. The main purpose of a SOC is to ensure continuous security monitoring and to identify suspicious activities before they cause damage to systems or data. SOC teams analyze system logs, monitor network traffic, and investigate alerts generated by security tools.

One of the primary objectives of a SOC is proactive threat detection, which means identifying potential threats before they become serious security incidents. SOC teams also handle incident response, which involves investigating security alerts, determining their impact, and taking appropriate action to mitigate the threat. Continuous monitoring of systems, networks, and applications helps organizations maintain strong security posture.

SOC teams usually consist of different roles based on expertise and responsibility are as follow:

- **Tier 1 analysts** are responsible for monitoring alerts and performing initial analysis of security events. They review alerts generated by security tools and determine whether the activity is suspicious.
- **Tier 2 analysts** perform deeper investigations by analyzing logs, identifying attack patterns, and correlating security events.
- **Tier 3 analysts** focus on advanced threat hunting and malware analysis to detect sophisticated attacks.
- **SOC manager** oversees the operations of the SOC team and ensures that security incidents are handled effectively.
- **Threat hunters** proactively search for hidden threats that may not trigger security alerts.

The key functions of a SOC include log analysis, alert triage, and threat intelligence integration.

- **Log analysis** involves examining system and network logs to detect unusual activities.
- **Alert triage** refers to prioritizing security alerts based on their severity and potential impact.

- **Threat intelligence integration** allows SOC analysts to use external threat data such as malicious IP addresses or domains to improve threat detection capabilities.

SOC analysts often follow security frameworks to guide their operations. Common frameworks include the **NIST cybersecurity framework**, which provides guidelines for managing cybersecurity risks, and the **MITRE ATT&CK framework**, which helps analysts understand attacker techniques and tactics. SOC training can also involve studying real-world SOC operations through case studies or walkthroughs from organizations such as IBM and Microsoft. In addition, workflow simulations using tools like **Splunk Phantom** can help analysts understand automated incident response processes.

2. Security Monitoring Basics

Security monitoring is a critical process in cybersecurity that involves continuously observing systems, networks, and applications to detect suspicious or unauthorized activities. The main goal of security monitoring is to identify potential security threats, policy violations, or abnormal system behavior that could indicate a cyber attack.

Security monitoring helps organizations detect events such as unauthorized login attempts, malware activity, and network intrusions. By analyzing security logs and network traffic, security analysts can identify patterns that indicate possible attacks. Early detection of these threats allows organizations to respond quickly and minimize potential damage.

Several tools are used for security monitoring. One of the most important tools is **Security Information and Event Management (SIEM)** systems, such as Splunk and Elastic. SIEM platforms collect logs from multiple sources, including servers, applications, and network devices, and analyze them to detect suspicious activities. These tools generate alerts when unusual patterns or security events are detected.

Another important tool used in security monitoring is **network traffic analyzers**, such as Wireshark. These tools capture and analyze network packets, allowing analysts to inspect communication between devices and identify malicious traffic patterns.

Security monitoring effectiveness is measured using several key metrics. One important metric is **false positives**, which occur when a security system generates an alert for activity that is not actually malicious. Another metric is **false negatives**, which occur when a real security threat is not detected by the monitoring system. Organizations also track **Mean Time to Detect (MTTD)**, which measures how quickly a security incident is identified after it occurs.

Effective security monitoring requires continuous analysis of logs, network traffic, and alerts to ensure that potential threats are identified and addressed promptly.

3. Log Management Fundamentals

Log management is an essential component of security operations because logs provide detailed records of activities occurring within systems, networks, and applications. Security analysts rely on logs to detect suspicious behavior, investigate incidents, and understand how attacks occur. Proper log management ensures that logs are collected, organized, stored, and analyzed efficiently.

The log management process follows a **log lifecycle**, which consists of several stages. The first stage is **log collection**, where logs are gathered from different sources such as operating systems, applications, and network devices. The second stage is **log normalization**, where logs from different sources are converted into a standard format to make analysis easier. The third stage is **log storage**, where logs are securely stored for further investigation. After storage, logs are kept for a specific period during the **log retention** phase, depending on organizational policies or compliance requirements. Finally, logs are examined during the **analysis stage**, where security analysts review the logs to identify suspicious or malicious activities.

Different systems generate different types of logs. One common type is **Windows Event Logs**, which record system activities such as user logins, application events, and security changes. These logs are often used to detect events such as failed login attempts or unauthorized system access. Another common log type is **Syslog**, which is widely used in Linux and network devices to record system messages, authentication events, and service activities. In addition, **HTTP server logs** record requests made to web servers and can help detect malicious web activity such as brute-force login attempts, scanning, or exploitation of web vulnerabilities.

In practical environments, log collection tools are used to gather logs from different systems. Tools such as **Fluentd** or **Logstash** can be configured to collect logs from servers and forward them to centralized monitoring platforms. During the normalization process, logs may be converted into structured formats such as **Common Event Format (CEF)** or **JSON**, which makes them easier to search and analyze.

Security analysts often query logs using search or query languages. For example, in security monitoring platforms such as **Azure Sentinel**, analysts can use **Kusto Query Language**

(KQL) to search and analyze log data. These queries allow analysts to filter events, identify patterns, and detect suspicious activities within large volumes of log data.
Effective log management helps organizations improve threat detection, investigate security incidents, and maintain visibility across their systems and networks. Proper collection, normalization, and analysis of logs enable security teams to quickly identify abnormal behavior and respond to potential security threats.

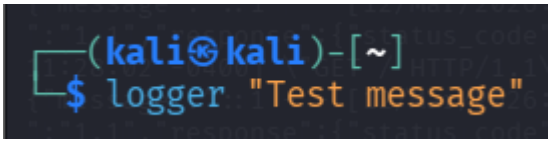
Practice Task

Log Collection Pipeline

A log collection pipeline was implemented to collect system logs using a log management tool. The system log service was configured to generate log entries and test the log collection process. A test log message was generated using the `logger` command in Kali Linux.
The following command was used to generate the test log:

logger "Test message"

The generated log entry was successfully captured by the logging service and verified in the system logs. This confirms that the log collection mechanism is functioning correctly and that logs can be monitored for security analysis.



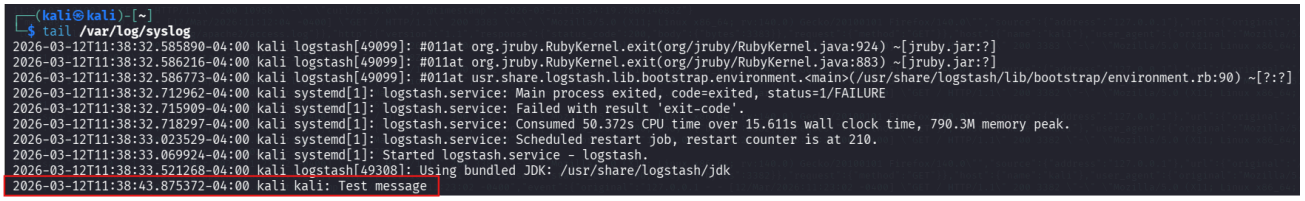


Figure 1: Verification of test log generated using the logger command

KQL Query Practice

The next writing a KQL (Kusto Query Language) query to detect failed login attempts in a SIEM platform. Failed login events in Windows systems are recorded with Event ID 4625.
The following query was used to detect failed login attempts and group them by source IP address:

source = "security-login-*"

EventID = 4625

| stats count by SourceIP

This query searches security login logs and filters events with Event ID 4625. The results are grouped by the source IP address to identify systems generating multiple failed login attempts. This helps SOC analysts detect possible brute-force attacks.

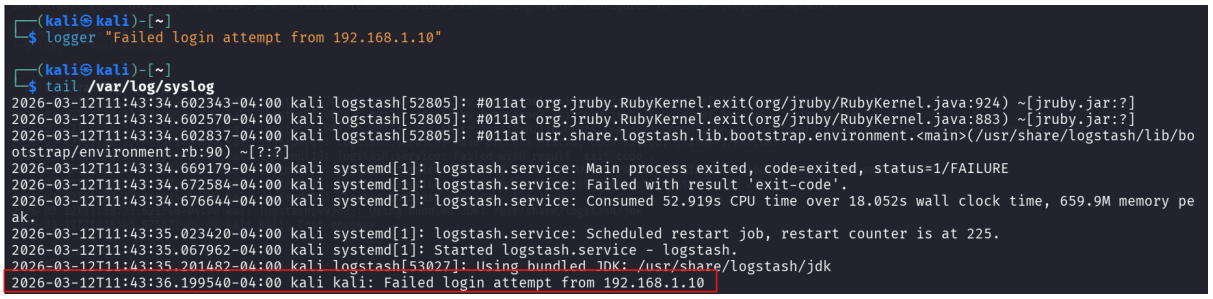


Figure 2: KQL query used to detect failed login attempts

Log Normalization Exercise

In Apache web server logs were converted into structured JSON format using Logstash. Apache logs are normally stored in an unstructured text format, which makes them difficult to analyze.

The Logstash pipeline was configured to read Apache access logs from the following file:

/var/log/apache2/access.log

A Grok filter was used to parse the raw log entries and extract important information such as the client IP address, request method, response status code, and user agent. The processed logs were then converted into JSON format and saved to a file for easier analysis.

This process is called **log normalization**, which helps security monitoring systems analyze log data more efficiently.

```
┌──(kali㉿kali)-[~]
└─$ cat /var/log/apache2/access.log
127.0.0.1 - - [12/Mar/2026:11:10:23 -0400] "GET / HTTP/1.1" 200 3383 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
40.0"
127.0.0.1 - - [12/Mar/2026:11:10:23 -0400] "GET /icons/openlogo-75.png HTTP/1.1" 200 6040 "http://localhost/" "Mozilla/5.0 (X11; Linux x86_6
4; rv:140.0) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [12/Mar/2026:11:10:23 -0400] "GET /favicon.ico HTTP/1.1" 404 527 "http://localhost/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0
) Gecko/20100101 Firefox/140.0"
127.0.0.1 - - [12/Mar/2026:11:11:34 -0400] "GET / HTTP/1.1" 200 3383 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
40.0"
127.0.0.1 - - [12/Mar/2026:11:11:43 -0400] "GET / HTTP/1.1" 200 3383 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
40.0"
127.0.0.1 - - [12/Mar/2026:11:12:04 -0400] "GET / HTTP/1.1" 200 3383 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
40.0"
127.0.0.1 - - [12/Mar/2026:11:12:19 -0400] "GET / HTTP/1.1" 200 3383 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
40.0"
::1 - - [12/Mar/2026:11:18:39 -0400] "GET / HTTP/1.1" 200 10958 "-" "curl/8.18.0"
::1 - - [12/Mar/2026:11:18:51 -0400] "GET / HTTP/1.1" 200 10958 "-" "curl/8.18.0"
::1 - - [12/Mar/2026:11:18:52 -0400] "GET / HTTP/1.1" 200 10958 "-" "curl/8.18.0"
::1 - - [12/Mar/2026:11:18:53 -0400] "GET / HTTP/1.1" 200 10958 "-" "curl/8.18.0"
127.0.0.1 - - [12/Mar/2026:11:20:38 -0400] "GET / HTTP/1.1" 200 3383 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
40.0"
```

Figure 3: Raw Apache access logs before normalization

```
┌──(kali㉿kali)-[~]
└─$ cat /home/kali/apache_logs.json
{"message":"127.0.0.1 - - [12/Mar/2026:11:11:34 -0400] \"GET / HTTP/1.1\" 200 3383 \"-\" \"Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
"log":{"file":{"path":"/var/log/apache2/access.log"}},"http":{"version":"1.1","response":{"status_code":200,"body":{"bytes":3383}},"request":{"method":"GET"}
64; rv:140.0) Gecko/20100101 Firefox/140.0"},"timestamp":"12/Mar/2026:11:11:34 -0400","event":{"original":"127.0.0.1 - - [12/Mar/2026:11:11:34 -0400] \"GET /
100101 Firefox/140.0\""},"@timestamp":"2026-03-12T15:34:19.743438267Z"}
{"message":"::1 - - [12/Mar/2026:11:18:53 -0400] \"GET / HTTP/1.1\" 200 10958 \"-\" \"curl/8.18.0\"","source":{"address":"::1"},"url":{"original":"/"},"@vers
":"1.1","response":{"status_code":200,"body":{"bytes":10958}},"request":{"method":"GET"}},"host":{"name":"kali"},"user_agent":{"original":"curl/8.18.0"},"tim
11:18:53 -0400] \"GET / HTTP/1.1\" 200 10958 \"-\" \"curl/8.18.0\""},"@timestamp":"2026-03-12T15:34:19.754789730Z"}
{"message":"::1 - - [12/Mar/2026:11:24:15 -0400] \"GET / HTTP/1.1\" 200 10958 \"-\" \"curl/8.18.0\"","source":{"address":"::1"},"url":{"original":"/"},"@vers
":"1.1","response":{"status_code":200,"body":{"bytes":10958}},"request":{"method":"GET"}},"host":{"name":"kali"},"user_agent":{"original":"curl/8.18.0"},"tim
11:24:15 -0400] \"GET / HTTP/1.1\" 200 10958 \"-\" \"curl/8.18.0\""},"@timestamp":"2026-03-12T15:34:19.760839711Z"}
{"message":"::1 - - [12/Mar/2026:11:31:02 -0400] \"GET / HTTP/1.1\" 200 10958 \"-\" \"curl/8.18.0\"","source":{"address":"::1"},"url":{"original":"/"},"@vers
":"1.1","response":{"status_code":200,"body":{"bytes":10958}},"request":{"method":"GET"}},"host":{"name":"kali"},"user_agent":{"original":"curl/8.18.0"},"tim
11:31:02 -0400] \"GET / HTTP/1.1\" 200 10958 \"-\" \"curl/8.18.0\""},"@timestamp":"2026-03-12T15:34:19.772171575Z"}
{"message":"127.0.0.1 - - [12/Mar/2026:11:22:57 -0400] \"GET / HTTP/1.1\" 200 3383 \"-\" \"Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/1
"log":{"file":{"path":"/var/log/apache2/access.log"}},"http":{"version":"1.1","response":{"status_code":200,"body":{"bytes":3383}},"request":{"method":"GET"}
64; rv:140.0) Gecko/20100101 Firefox/140.0"},"timestamp":"12/Mar/2026:11:22:57 -0400","event":{"original":"127.0.0.1 - - [12/Mar/2026:11:22:57 -0400] \"GET /
100101 Firefox/140.0\""},"@timestamp":"2026-03-12T15:34:19.756149887Z"}
```

Figure 4: Apache logs converted into structured JSON format

Result

The practical tasks demonstrated the process of log collection, security event detection, and log normalization. System logs were generated and verified using Linux logging utilities. A query was written to detect failed login attempts in a SIEM environment, and Apache web server logs were successfully converted into structured JSON format using Logstash. These tasks illustrate the importance of log management in security monitoring and incident investigation.

4. Security Tools Overview

Security Operations Centers (SOC) rely on various security tools to monitor systems, detect threats, and respond to security incidents. These tools help analysts analyze logs, detect malicious activities, and identify vulnerabilities within the network infrastructure.

One of the most important tools used in SOC environments is a **Security Information and Event Management (SIEM)** system. SIEM platforms such as Splunk and IBM QRadar collect logs from multiple sources including servers, applications, and network devices. These logs are analyzed in real time to detect suspicious activities and generate security

alerts. SIEM systems also allow security analysts to search logs, investigate incidents, and create dashboards for security monitoring.

Another important category of security tools is **Endpoint Detection and Response (EDR)**. EDR solutions monitor activities occurring on endpoint devices such as computers and servers. An example of an EDR tool is CrowdStrike Falcon. These tools help detect malicious processes, unauthorized access, and malware activity on endpoint systems. EDR tools also provide threat detection, investigation, and response capabilities.

Network security is often monitored using **Intrusion Detection and Prevention Systems (IDS/IPS)**. These systems analyze network traffic to detect malicious activities such as attacks, scanning, or exploitation attempts. One widely used open-source IDS tool is Snort. IDS tools detect suspicious network patterns, while IPS systems can automatically block malicious traffic.

Another essential security tool is a **vulnerability scanner**, which helps identify security weaknesses in systems and applications. Vulnerability scanners analyze systems to detect outdated software, misconfigurations, and security vulnerabilities. One commonly used vulnerability scanning tool is Nessus. By identifying vulnerabilities early, organizations can apply patches and security controls before attackers exploit these weaknesses.

Practical Task

Snort Rule Testing

Description

In this task, the network intrusion detection system Snort was used to create a custom rule to detect HTTP requests containing a suspicious domain. Snort analyzes network traffic and generates alerts when predefined rules match specific patterns.

Snort Rule

The following rule was written to detect HTTP requests to the domain **malicious.com**:

```
alert tcp any any -> any 80 (msg:"Malicious Domain"; content:"malicious.com"; http_uri; sid:1000001;)
```

Testing the Rule

The rule was tested using the following command:

```
curl http://malicious.com
```

When the request was detected, Snort generated an alert indicating that a request to the malicious domain had been identified.

```
┌──(kali㉿kali)-[~]
└─$ sudo snort -A alert_fast -c /etc/snort/snort.lua -i eth0
[sudo] password for kali:
--------------------------------------------------
o")~   Snort++ 3.11.1.0
--------------------------------------------------
Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
        active
        alerts
        daq
        decode
        host_cache
        hosts
        network
        process
```

```
Session  Actions  Edit  View  Help
        match list memory: 28.4375
        transition memory: 22.9453
appid: MaxRss diff: 3072
appid: patterns loaded: 300
--------------------------------------------------
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] eth0
03/13-01:07:06.630273 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:20.610453 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:21.964326 [**] [122:15:1] "(port_scan) IP filtered protocol sweep" [**] [Priority: 3] {IP} 192.168.52.141 → 8.8.8
03/13-01:07:22.210147 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:22.274148 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:22.702284 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:23.195254 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:23.255272 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:24.592566 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:24.592629 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:24.603432 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:24.603448 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
03/13-01:07:24.604683 [**] [116:6:1] "(ipv4) IPv4 datagram length > captured length" [**] [Priority: 3] {eth}  →
```

Figure 5: Snort detecting HTTP request to malicous domain

Nessus Vulnerability Scan

Description

A vulnerability scan was performed using Nessus Essentials to identify security weaknesses in the target system. Vulnerability scanners analyze services, configurations, and software versions to detect known vulnerabilities.

Scan Configuration

The **Basic Network Scan** template was used in Nessus to scan the target system. After the scan completed, the vulnerabilities were sorted based on their CVSS scores.

Top Vulnerabilities Detected

Vulnerability	CVSS Score	Description

Samba Badlock Vulnerability	7.5	Vulnerability affecting Samba authentication.
ICMP Timestamp Request Disclosure	2.1	Allows attackers to obtain system time information.
SMB (Multiple Issues)	-	Indicates possible configuration or security weaknesses in SMB services.

Sev	CVSS	VPR	EPSS	Name	Family	Count
INFO	7.5			Samba Badlock Vulnerability	General	1
INFO	2.1 *			ICMP Timestamp Request Remote Date Discl...	General	1
INFO	...	...	...	5 SMB (Multiple Issues)	Windows	6
INFO	...	...	...	5 ISC Bind (Multiple Issues)	DNS	5
INFO	...	...	...	4 DNS (Multiple Issues)	DNS	4
INFO	...	...	...	2 RPC (Multiple Issues)	RPC	2
INFO				Nessus SYN scanner	Port scanners	24
INFO				RPC Services Enumeration	Service detection	10
INFO				NFS Share Export List	RPC	1

Figure 6: Nessus vulnerability scan results

Osquery Monitoring

Description

Endpoint monitoring was performed using Osquery on a Windows system. Osquery allows security analysts to query system information using SQL-like queries.

Process Query

The following query was used to display running processes:

SELECT pid, name, cmdline FROM processes WHERE name='cmd.exe';

Suspicious Process Simulation

A batch file (`test.bat`) was executed to simulate a suspicious process. Since Windows executes batch files through `cmd.exe`, the process appeared as `cmd.exe` in the Osquery results with the command line:

cmd.exe /c "D:\test.bat"

This demonstrates how endpoint monitoring tools can detect suspicious or unknown processes.

```
osquery> SELECT pid, name, cmdline FROM processes WHERE name = 'cmd.exe';
+-------+---------+-------------------------------------------------+
| pid   | name    | cmdline                                         |
+-------+---------+-------------------------------------------------+
| 19208 | cmd.exe | "C:\WINDOWS\system32\cmd.exe"                   |
| 18176 | cmd.exe | C:\WINDOWS\system32\cmd.exe /c ""D:\test.bat" " |
+-------+---------+-------------------------------------------------+
```

Figure 7: Osquery detecting execution of a batch script through cmd.exe

5. Basic Security Concepts

Basic security concepts form the foundation of cybersecurity practices and help organizations protect their information systems from threats and attacks. Understanding these concepts enables security analysts to identify risks, implement security controls, and respond effectively to security incidents.

One of the fundamental principles of information security is the **CIA Triad**, which consists of **Confidentiality, Integrity, and Availability**. Confidentiality ensures that sensitive information is accessible only to authorized individuals and protected from unauthorized disclosure. Integrity ensures that data remains accurate and unaltered unless modified by authorized users. Availability ensures that systems, applications, and data remain accessible to authorized users whenever they are needed.

Another important concept in cybersecurity is the relationship between **threats, vulnerabilities, and risks**. A **threat** refers to a potential cause of harm to a system or

network, such as hackers, malware, or insider attacks. A **vulnerability** is a weakness in a system, application, or configuration that could be exploited by an attacker. **Risk** represents the potential damage or impact that may occur when a threat exploits a vulnerability. Security professionals assess risks to determine appropriate mitigation strategies.

Modern cybersecurity strategies also rely on security models such as **defense-in-depth** and **zero trust architecture**. Defense-in-depth is a layered security approach where multiple security controls are implemented at different levels of the system to protect against attacks. This may include firewalls, intrusion detection systems, authentication mechanisms, and endpoint protection. Zero trust is a security model that assumes no user or device should be automatically trusted, even if they are inside the network. Instead, every access request must be verified and authenticated before access is granted.

Real-world security incidents highlight the importance of these security concepts. For example, the Equifax Data Breach demonstrated how an unpatched vulnerability can lead to large-scale data exposure. Attackers exploited a vulnerability in a web application framework, resulting in the exposure of sensitive personal data of millions of individuals.

6. Security Operations Workflow

Security Operations Centers (SOC) follow a structured workflow to detect, analyze, and respond to cybersecurity incidents. This workflow helps security analysts systematically investigate alerts, determine the severity of threats, and take appropriate actions to protect organizational systems.

The first stage of the security operations workflow is **detection**. In this stage, security monitoring tools generate alerts when suspicious activities are identified. Tools such as Splunk or CrowdStrike Falcon analyze logs, network traffic, and endpoint activities to detect unusual behavior. These alerts notify SOC analysts about potential security incidents that require investigation.

The second stage is **triage**, where analysts review and prioritize alerts based on their severity and potential impact. Not all alerts represent real threats; some may be false positives generated by monitoring systems. During triage, analysts evaluate the alert details and determine whether the activity requires further investigation.

The next stage is **investigation**. In this stage, security analysts analyze logs, correlate events from multiple systems, and search for indicators of compromise (IOCs). Log data from SIEM systems, endpoint monitoring tools, and network devices are examined to understand the scope and nature of the potential attack. Analysts may also compare observed activities with known attacker techniques using frameworks such as MITRE ATT&CK.

The final stage is **response**, where actions are taken to contain and eliminate the threat. This may include blocking malicious IP addresses, isolating compromised systems, removing malicious files, or resetting compromised user accounts. After the threat has been mitigated, systems are restored to normal operation and preventive measures are implemented to avoid similar incidents in the future.

Security analysts often simulate incident response workflows using platforms such as TheHive. These simulations help analysts practice responding to incidents such as phishing attacks, malware infections, or unauthorized access attempts.

Understanding the security operations workflow enables SOC teams to efficiently detect threats, investigate suspicious activities, and respond quickly to security incidents, thereby improving the organization's overall security posture.

7. Incident Response Basics

Incident response is a structured approach used by organizations to detect, investigate, and respond to cybersecurity incidents. The goal of incident response is to minimize damage, reduce recovery time, and prevent similar incidents from occurring in the future. Security Operations Centers follow established frameworks to ensure incidents are handled in a systematic and efficient manner.

One widely recognized framework for incident response is the guideline provided in NIST SP 800-61. This framework defines a lifecycle consisting of several stages that guide organizations in responding to cybersecurity incidents.

The first stage is **preparation**, where organizations establish policies, procedures, and tools required for incident handling. This includes deploying monitoring tools, defining incident

response teams, and creating response playbooks to guide analysts during security incidents.

The second stage is **identification**, where security analysts detect and confirm the presence of a security incident. Alerts generated by monitoring systems such as SIEM or endpoint protection tools are analyzed to determine whether suspicious activity represents a genuine threat.

Once an incident is confirmed, the next stage is **containment**. In this phase, steps are taken to prevent the threat from spreading further within the network. This may involve isolating infected systems, blocking malicious IP addresses, or disabling compromised user accounts.

After containment, the **eradication** stage focuses on removing the root cause of the incident. Malicious files, unauthorized access points, or vulnerabilities exploited by attackers are eliminated to ensure the threat no longer exists within the environment.

The next stage is **recovery**, where affected systems are restored to normal operation. Systems may be restored from backups, patched, and monitored to ensure they are functioning securely.

The final stage is **lessons learned**, where the incident response team reviews the incident to understand how it occurred and how future incidents can be prevented. Improvements may include updating security policies, strengthening monitoring rules, or applying additional security controls.

Organizations often conduct simulated incident response exercises, such as ransomware attack scenarios, to train security teams and improve their readiness to handle real-world cyber incidents. These exercises help analysts practice decision-making, communication, and technical response procedures in a controlled environment.

8. Documentation Standards

Proper documentation is an essential part of cybersecurity operations. Security teams must maintain detailed records of incidents, procedures, and investigations to ensure consistency, accountability, and effective knowledge sharing. Well-structured documentation helps

organizations track security events, improve incident response processes, and support future investigations.

One of the most common types of documentation is the **incident report**. An incident report records important details about a security event, including the time of occurrence, affected systems, indicators of compromise, and actions taken to resolve the incident. These reports provide a clear record of how the incident was handled and are often used for internal review and compliance purposes.

Another important document used in security operations is a **runbook**. Runbooks contain step-by-step instructions for handling specific types of incidents, such as phishing attacks, malware infections, or unauthorized access attempts. Runbooks help security analysts respond quickly and consistently to security alerts without needing to design a response process from scratch.

Organizations also maintain **Standard Operating Procedures (SOPs)**, which define the standard processes and responsibilities for routine security operations. SOPs ensure that security tasks such as log monitoring, vulnerability scanning, and incident handling are performed consistently across the organization.

After a security incident has been resolved, security teams often conduct a **post-mortem analysis**. This process reviews the incident in detail to identify its root cause, evaluate the effectiveness of the response, and determine how similar incidents can be prevented in the future. Post-mortem reports typically include recommendations for improving security controls, updating response procedures, and strengthening monitoring systems.

Security professionals often follow structured documentation guidelines such as those provided in the SANS Incident Handler's Handbook. These templates help ensure that incident documentation includes all necessary information, such as event timelines, analysis findings, and remediation actions.

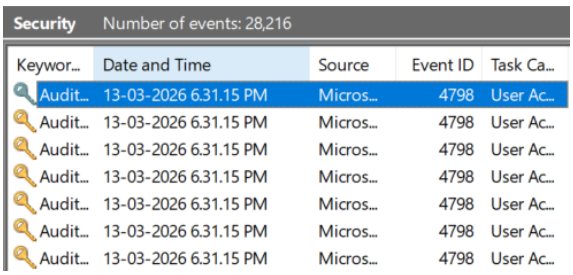
Maintaining clear and consistent documentation improves communication among security teams, supports compliance requirements, and helps organizations continuously improve their cybersecurity posture.

Practical Application

1.Log Analysis Practice

Windows Event Viewer was used to analyze security logs and detect suspicious authentication events. The Security log was filtered using **Event ID 4625**, which represents failed login attempts. These events are useful for identifying possible brute-force attacks where an attacker repeatedly attempts to log in with incorrect credentials.

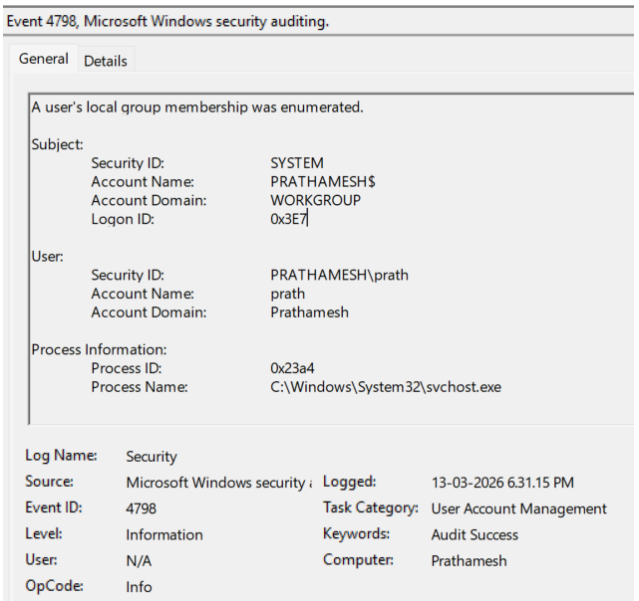
To simulate the scenario, several incorrect login attempts were performed on the Windows system. These attempts generated failed login events in the Security log, which were then filtered using Event Viewer.



Security Number of events: 28,216

Keywor...	Date and Time	Source	Event ID	Task Ca...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...

Figure 8: Windows Event Viewer showing filtered Security logs with Event ID 4625 (failed login attempts).



Event 4798, Microsoft Windows security auditing.

General Details

A user's local group membership was enumerated.

Subject:
Security ID: SYSTEM
Account Name: PRATHAMESH$
Account Domain: WORKGROUP
Logon ID: 0x3E7

User:
Security ID: PRATHAMESH\prath
Account Name: prath
Account Domain: Prathamesh

Process Information:
Process ID: 0x23a4
Process Name: C:\Windows\System32\svchost.exe

Log Name: Security
Source: Microsoft Windows security
Logged: 13-03-2026 6.31.15 PM
Event ID: 4798
Task Category: User Account Management
Level: Information
Keywords: Audit Success
User: N/A
Computer: Prathamesh
OpCode: Info

Figure 9: Detailed view of Event ID 4625 showing information about a failed login attempt.

2. Document Security Events

Proper documentation of security events is important for tracking incidents, analyzing attack patterns, and maintaining records for future investigation. Security analysts document important details such as the time of the event, source of the activity, type of event detected, and the response actions taken.

During this task, a mock security event was documented to demonstrate how security incidents are recorded in a Security Operations Center (SOC).

Security Event Documentation Template

The following template was used to record the event details:

Date/Time	Source IP	Event ID	Description	Action Taken
13-03-2026 14:25	192.168.1.10	4625	Multiple failed login attempts detected in Windows Security logs	Account monitored and further login attempts blocked

Event Description

The security logs indicated several failed login attempts associated with **Event ID 4625**, which represents a failed authentication attempt in Windows systems. Multiple login failures occurring within a short period may indicate a potential brute-force attack.

The event was recorded using the documentation template to capture important details including the source IP address, event ID, and the response action taken by the security analyst.

Maintaining such documentation helps security teams track suspicious activities, analyze security incidents, and improve response procedures in future investigations.

3. Set Up Monitoring Dashboards

Monitoring dashboards are used in Security Operations Centers (SOC) to visualize security events and detect abnormal activity within a network. Dashboards help security analysts quickly identify patterns, investigate alerts, and prioritize potential threats by presenting log data in graphical form.

Security monitoring tools such as Kibana and Grafana are commonly used to create dashboards that visualize security logs collected from different systems. These dashboards display important metrics such as alert activity, event frequency, and source IP addresses generating suspicious activity.

The monitoring dashboard was created using exported log data. The dashboard visualizes security alerts generated by different source IP addresses and the frequency of important security event IDs.

Top Source IPs Generating Alerts

The first visualization displays the top source IP addresses responsible for generating security alerts. This helps analysts identify systems that may be attempting unauthorized access or generating suspicious activity within the network.

The chart shows that the IP address **127.0.0.1** generated the highest number of alerts, followed by **192.168.52.141** and **192.168.52.131**. Monitoring such activity allows analysts to investigate systems responsible for repeated alerts and determine whether they represent potential threats.

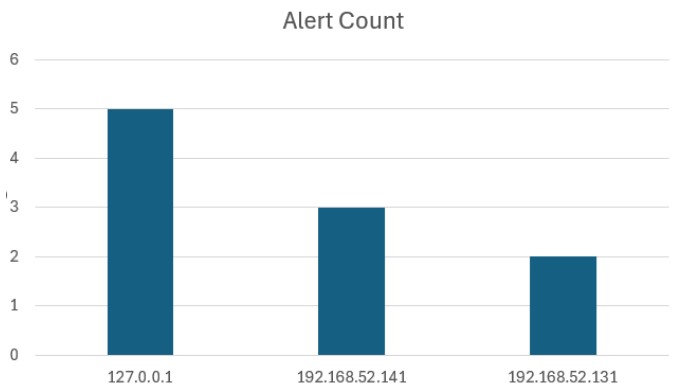


Figure 10: Monitoring dashboard showing top source IP addresses generating security alerts

Frequency of Security Event IDs

The second visualization represents the frequency of security event IDs detected in the system logs. Event IDs provide important information about security-related activities occurring within a system.

The analysis focused on the following event IDs:

Event ID	Description
4625	Failed login attempt
7045	New service installation
4688	Process creation

Among these events, **Event ID 4625** occurred most frequently, indicating multiple failed login attempts. Such activity may indicate a potential brute-force login attempt or unauthorized access attempts.

Visualizing these events helps security analysts quickly identify abnormal patterns in authentication activity and system behavior.

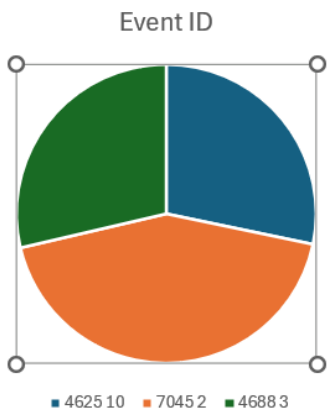


Figure 11: Frequency distribution of security Event IDs detected in system logs

4. Configure Alert Rules

Alert rules are used in Security Information and Event Management (SIEM) systems to automatically detect suspicious activities based on predefined conditions. When certain thresholds or patterns are detected in system logs, the monitoring platform generates alerts to notify security analysts about potential threats.

Security monitoring platforms such as Elastic Security allow analysts to define custom rules that monitor authentication events and trigger alerts when abnormal behavior occurs.

Alert Rule Configuration

In this task, an alert rule was designed to detect multiple failed login attempts within a short period of time, which may indicate a brute-force attack.

Parameter	Configuration
Rule Name	Detect Multiple Failed Logins
Log Index	security-login-*
Event Type	Failed login events
Condition	More than 5 failed logins
Time Window	5 minutes

The rule monitors authentication logs and generates an alert when the number of failed login attempts exceeds the defined threshold.

Rule Testing Using System Logs

To simulate this scenario, multiple incorrect login attempts were performed on the Windows system. These attempts generated **Event ID 4625** entries in the Windows Security logs, which represent failed authentication attempts.

The log analysis revealed that **10 failed login attempts** occurred within a short time period. If the alert rule had been configured in a SIEM system, this activity would have triggered an alert because the number of failed login attempts exceeded the threshold of five attempts within five minutes.

Result

The analysis confirms that repeated failed login attempts were successfully recorded in the system logs. Such activity could trigger automated alerts in security monitoring platforms and help security analysts quickly detect potential brute-force attacks.

Security Number of events: 28,216

Keywor...	Date and Time	Source	Event ID	Task Ca...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...
Audit...	13-03-2026 6.31.15 PM	Micros...	4798	User Ac...

Figure 12: Windows Security logs showing multiple failed login attempts (Event ID 4798) that satisfy the alert rule condition

Key Learnings

During this task, practical knowledge of security monitoring and log analysis was gained. The exercises demonstrated how system logs can be analyzed to detect suspicious activities such as repeated failed login attempts and potential brute-force attacks. Practical experience was also gained in using security tools for intrusion detection, vulnerability scanning, endpoint monitoring, and visualizing security events through monitoring dashboards. These activities highlighted the importance of automated alert rules and proper documentation in improving the effectiveness of Security Operations Center (SOC) monitoring and incident response.